

Hardware & Software Verification

John Wickerson

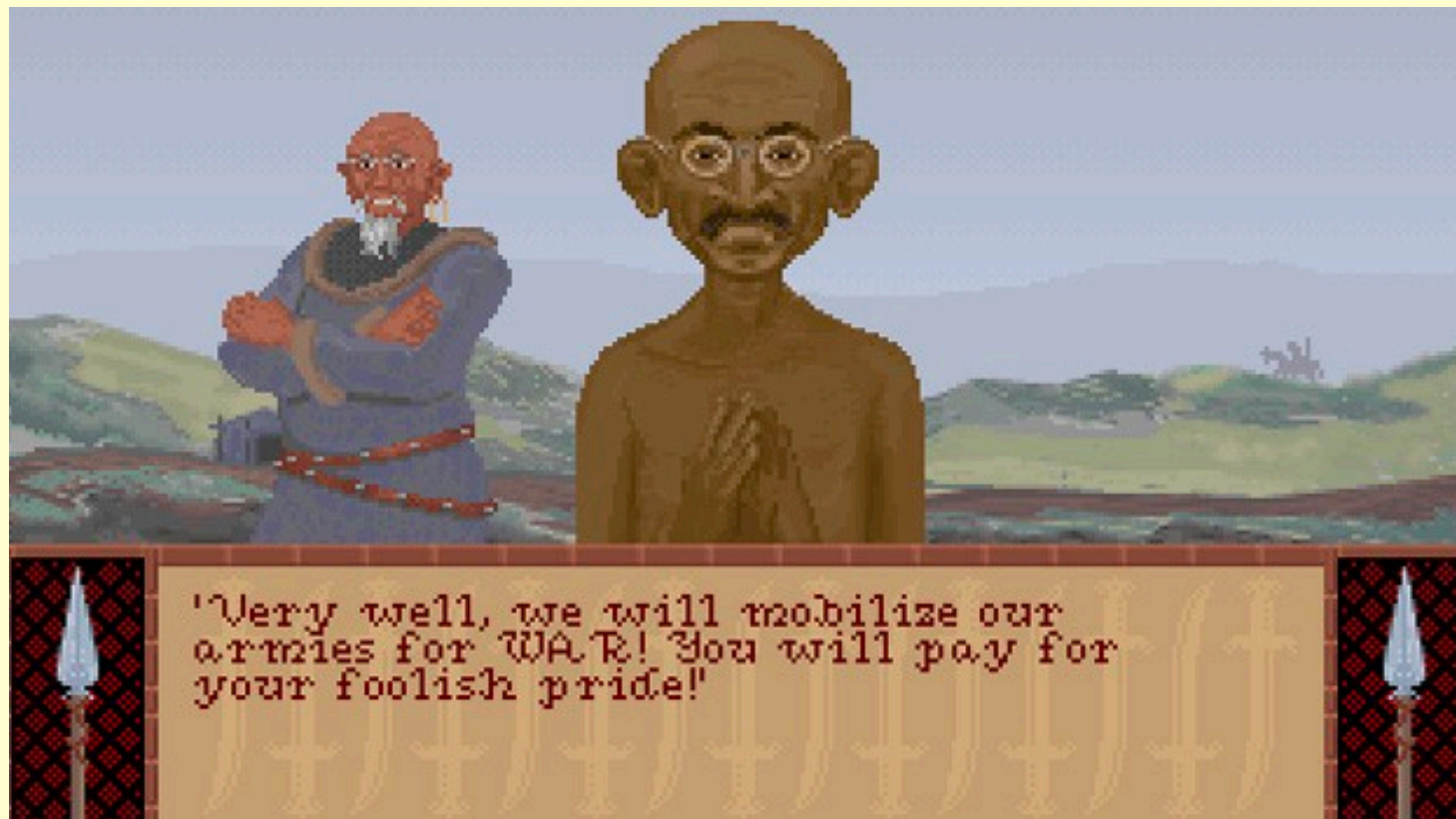
<https://github.com/johnwickerson/HSV>

Lecture 1
5 October 2026

The correctness problem



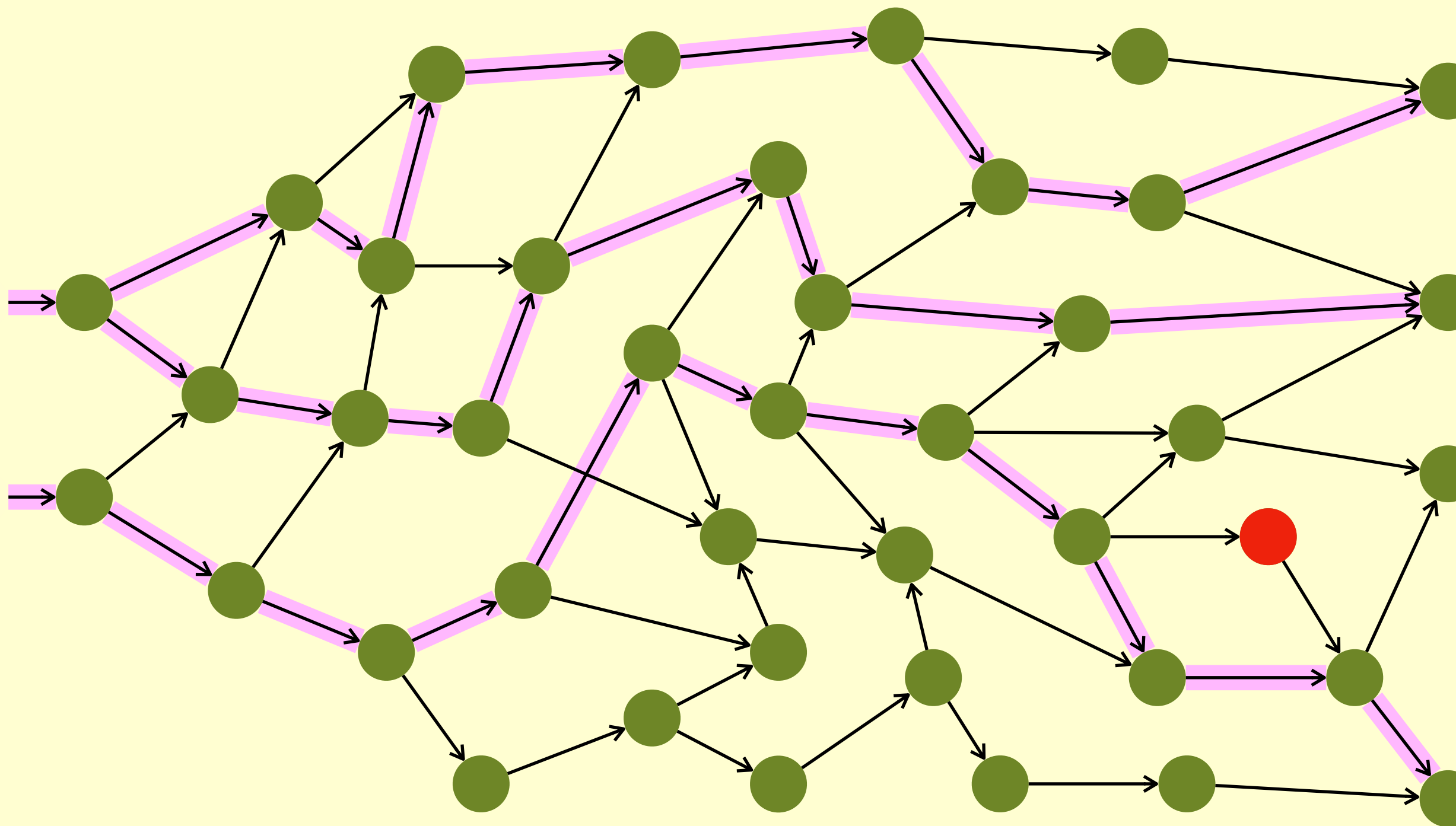
```
int main () {  
    double a = 4195835;  
    double b = 3145727;  
    printf ("%f\n", a/b);  
}
```



The correctness problem

- Computer systems are becoming more **complicated** and more **trusted**.
- This means that being confident that they are correct is increasingly **difficult** and increasingly **important**.
- Traditional **testing** is no longer enough, especially in our manycore era.
- Fortunately there are techniques and tools for **verifying** that a computer system is correct.

The problem with testing



The problem with testing

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char **argv) {
    if (argc > 1 && atoi(argv[1]) == 4242) {
        printf ("KABOOM!\n");
        return 1;
    }
}
```

The problem with testing

```
int main() {  
    int x = 0, y = 0;  
    int r1 = 0, r2 = 0;  
  
    x = 1;    ||    r1 = y;  
    y = 1;    ||    r2 = x;  
  
    if (r1==1 && r2==0) {  
        printf ("KABOOM!\n");  
        return 1;  
    } else {  
        printf ("r1=%d, r2=%d\n", r1, r2);  
        return 0;  
    }  
}
```

- Doesn't cope with **nondeterminism** (e.g. from concurrency).
- Testing results may not be **portable**.

Can we do better?

- Rather than testing on many *particular* inputs, construct a general argument for why the program is correct on *all* inputs.



"There is no solution to $a^n + b^n = c^n$ when $n > 2$."

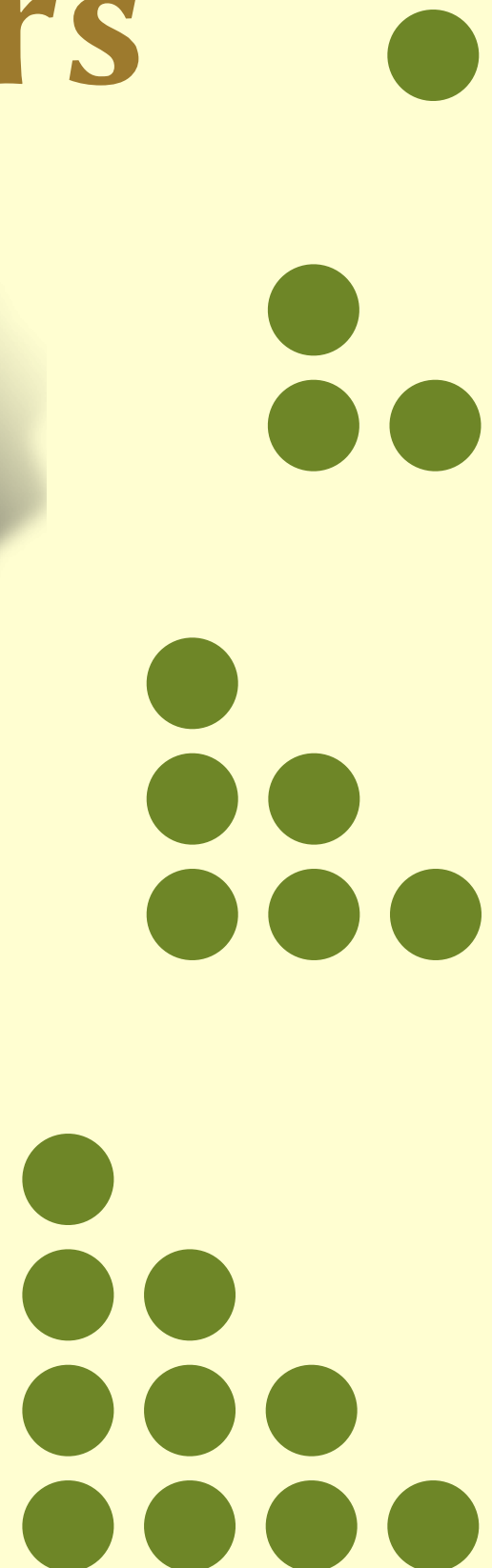


Triangle numbers

```
#include <stdlib.h>
#include <assert.h>

int triangle(int n) {
    int t = 0, i = 0;
    while (i < n) {
        i = i+1;
        t = t+i;
    }
    assert(t == n * (n+1) / 2);
    return t;
}

int main(int argc, char **argv) {
    int n = atoi(argv[1]);
    printf("%d\n", triangle(n));
}
```



Dafny

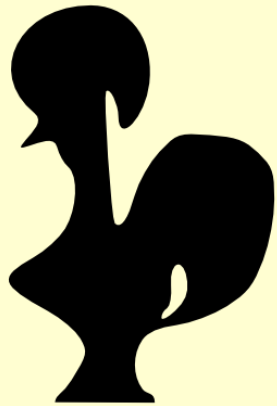
- Developed since 2008 by Rustan Leino and others at Microsoft Research, and nowadays mainly at Amazon.



Verifying with Dafny

```
method triangle(n:int) returns (t:int)
  requires 0 <= n
  ensures t == n * (n+1) / 2
{
  t := 0;
  var i := 0;
  while i < n
    invariant t == i * (i+1) / 2
    invariant i <= n
  {
    i := i+1;
    t := t+i;
  }
  return t;
}
```

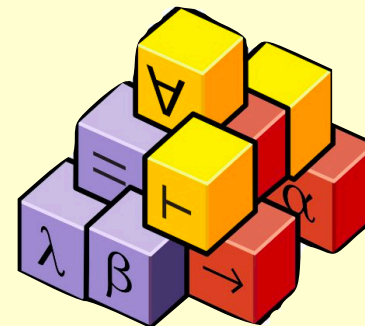
Interactive theorem proving



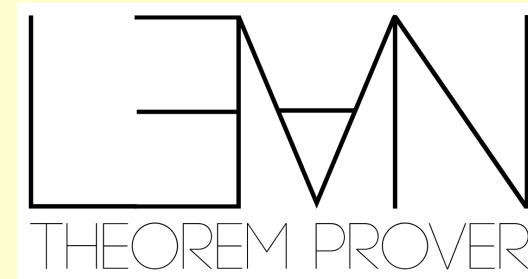
Rocq



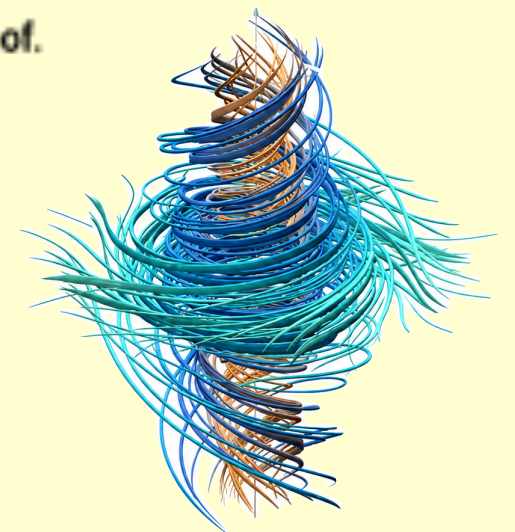
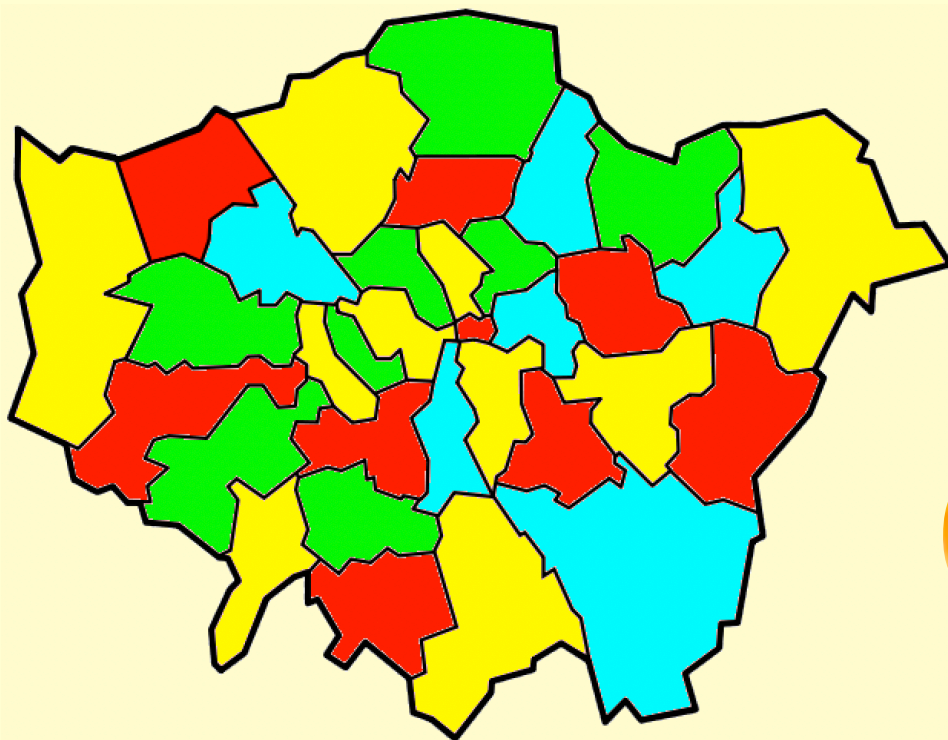
HOL



Isabelle

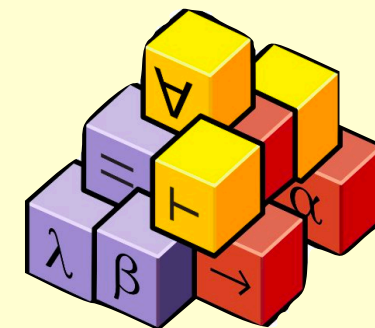


Lean

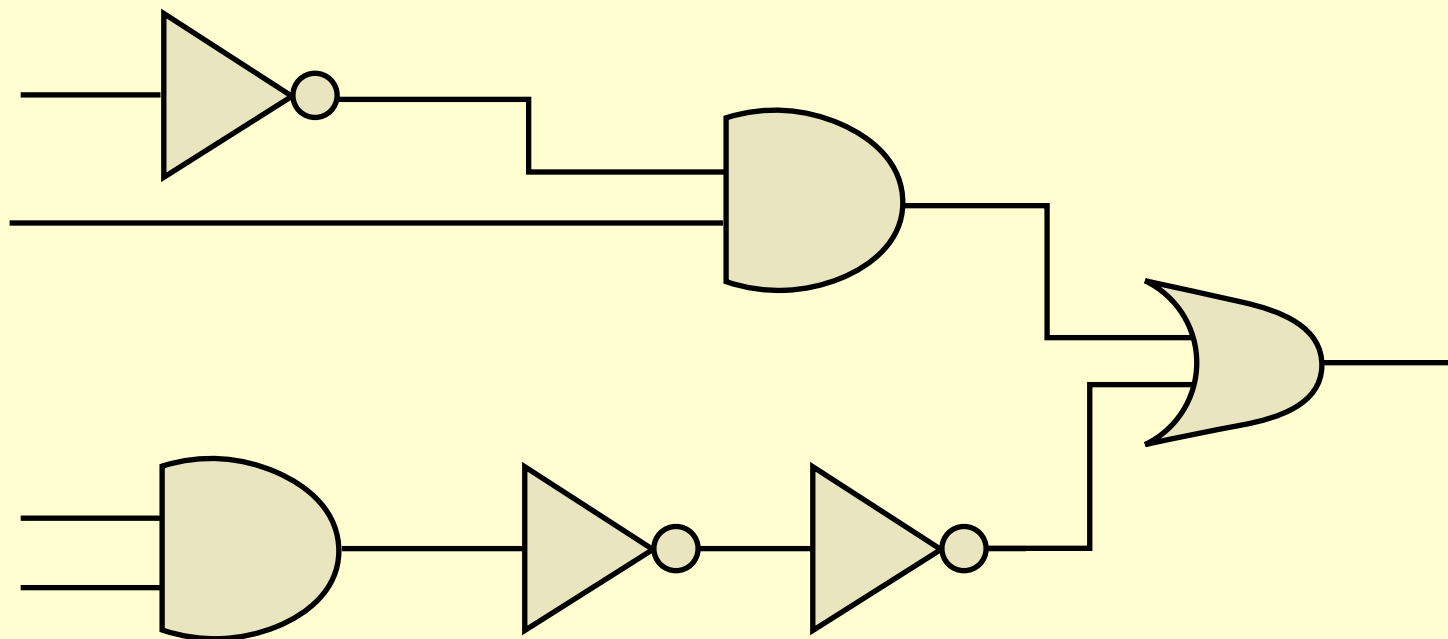


Interactive theorem proving

- *Your task:* prove a correctness theorem about a little logic synthesiser.



Isabelle



Computer-aided proof

Fully
manual

Fully
automatic

Rocq

SymbiYosys

Facebook Infer

HOL

Dafny

Astrée

Isabelle

VCC

Model checking

Aims of the module

1. Be able to use Dafny to verify simple programs.
 - **Assessment:** I will give you several (increasingly difficult) programs, and you have to verify them using Dafny.
2. Be able to use Isabelle to conduct simple mathematical proofs.
 - **Assessment:** I will give you several (increasingly difficult) theorems, and you have to prove them using Isabelle.
3. Be able to use SymbiYosys to verify simple Verilog designs.
 - **Assessment:** I will give you several (increasingly difficult) Verilog designs, and you have to verify them using SymbiYosys.

Rough plan

- Introduction
- Proving theorems using Isabelle
- Verifying software using Dafny
- Verifying hardware using SymbiYosys
- Principles of verification
- **Guest lecture from Apple:** date TBC.

Rough plan

w.b.	Monday 4-6pm	Thursdays 12-2pm
5 Oct	Introduction	Isabelle 1
12 Oct	Isabelle 2	<i>Lab</i>
19 Oct	Dafny	<i>Lab</i>
26 Oct	Formal HW verification	Guest lecture from Apple
2 Nov	SAT / SMT	Temporal logic and BDDs
9 Nov	Mock test (room 303)	<i>Lab</i>
16 Nov	Hoare Logic	<i>Lab</i>
23 Nov	Test: Isabelle (room 303)	<i>Lab</i>
30 Nov	Test: Dafny (room 303)	<i>Lab</i>
7 Dec	Test: Symbiosys (room 303)	Free!

Teaching support



A first proof

- **Theorem.** $\sqrt{2}$ is irrational.

A first proof

- **Theorem.** $\sqrt{2}$ is irrational.
- **Proof.** We proceed by contradiction. Suppose $\sqrt{2}$ is rational. Then we can obtain integers m and n such that $\sqrt{2} = m/n$, where $n > 0$ and $\gcd(m, n) = 1$. Squaring both sides gives us $2 = m^2/n^2$, and hence $2n^2 = m^2$. This means that m^2 is even, and hence so is m . Hence, we can obtain an integer m' such that $m = 2m'$. It follows that $2n^2 = (2m')^2$, and hence $n^2 = 2m'$. This means that n^2 is even, and hence so is n . But if both m and n are even, their gcd is at least 2, which contradicts the fact that $\gcd(m, n) = 1$. Thus our original supposition must have been wrong, which means that $\sqrt{2}$ is *not* rational.



On AI

- AI is having huge impact on formal verification
- AI can generate Isabelle proofs, and the proof checker means we don't have to worry about hallucinations!
- There is suddenly a huge **demand** for proofs *and* a huge **supply** of them!
- We still need humans to check the *definitions*.
- Feel free to use AI to help you learn, but no AI in the tests!